

Лабораторная работа №7

Компьютерный практикум по статистическому анализу данных

Канева Екатерина, НФИбд-02-22

6 декабря 2025

Российский университет дружбы народов, Москва, Россия

Информация

- Канева Екатерина Павловна
- студент группы НФИбд-02-22
- Российский университет дружбы народов
- 1132222004@rudn.ru
- <https://nevseros.github.io/ru/>

Вводная часть

Основной целью работы является освоение специализированных пакетов Julia для обработки данных.

- Используя Jupyter Lab, повторить примеры.
- Выполнить задания для самостоятельной работы.

Выполнение работы

Примеры

Примеры со считыванием данных

Выполнила примеры со считыванием данных:

```
[7]: function language_created_year(P, language::String)
      loc = findfirst(P[:,2].==language)
      return P[loc,1]
    end

[7]: language_created_year (generic function with 1 method)

[8]: # Пример вызова функции и определение даты создания языка Python:
      language_created_year(P, "Python")

[8]: 1991

[11]: # Пример вызова функции и определение даты создания языка Julia:
      language_created_year(P, "Julia")

[11]: 2012

[15]: # Функция определения по названию языка программирования
      # года его создания (без учета регистра):
      function language_created_year_v2(P, language::String)
        loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
        return P[loc,1]
      end

[15]: language_created_year_v2 (generic function with 1 method)

[18]: language_created_year_v2(P, "Julia")

[18]: 2012
```

Рис. 1: Примеры со считыванием данных.

Выполнила примеры со словарями:

```
[25]: # Сохраняю словарь в CSV-файл:  
CSV.write("programming_languages_data2.csv", P)  
  
[26]: "programming_languages_data2.csv"  
  
[27]: # Инициализирую словарь:  
dict = Dict{Integer,Vector{String}}()  
  
[27]: Dict{Integer, Vector{String}}()  
  
[28]: # Заполню словарь данными:  
for i = 1:size(P,1)  
    year,lang = P[i,:]  
    if year in keys(dict)  
        dict[year] = push!(dict[year],lang)  
    else  
        dict[year] = [lang]  
    end  
end  
  
[28]: dict{2003}  
  
[29]: 2-element Vector{String}:  
 "Groovy"  
 "Scala"
```

Рис. 2: Примеры со словарями.

Примеры с missing типом

Выполнила примеры с missing типом:

```
[39]: # Определить тип переменных:
      typeof(iris)

[39]: DataFrame

[40]: describe(iris)

[40]: 5x7 DataFrame
      Row variable  mean  min  median  max  nmissing  eltype
      Symbol  Union... Any  Union... Any  Int64  DataType
1 SepalLength  5.84333  4.3  5.8  7.9  0  Float64
2 SepalWidth  3.05733  2.0  3.0  4.4  0  Float64
3 PetalLength  3.758  1.0  4.35  6.9  0  Float64
4 PetalWidth  1.19933  0.1  1.3  2.5  0  Float64
5 Species      setosa  virginica  0  Categorical{String, Union{}}
```

```
[41]: # Определить тип missing
      a = missing
      typeof(a)

[41]: Missing

[42]: a + 1

[42]: missing
```

Рис. 3: Примеры с missing типом.

Выполнила примеры с join:

```
[50]: # Задание: объединить в таблицу:  
prices = [0.85,1.6,0.8,0.6]  
# форматирование данных в таблицу:  
dataframe_calories = DataFrame(item=foods,calories=calories)  
# форматирование данных в таблицу:  
dataframe_prices = DataFrame(item=foods,price=prices)  
# Объединение данных в таблицу и вывод:  
DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)
```

```
[59]: 4x3 DataFrame
```

Row	item	calories	price
	String	Int64?	Float64
1	apple	missing	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Рис. 4: Примеры с join.

Выполнила примеры со считыванием картинки:

```
[60]: # Загрузка изображения:  
X1 = load("julia.png")  
  
[60]:  
  
  
Warning: Output swatches are reduced due to the large size (180x280).  
Load the ImageShow package for large images.  
@ Colors: C:\Users\ekaneva\julia\packages\Colors\WEJ1\src\display.jl:159  
  
[71]: # Определения типа и размера данных:  
@show typeof(X1);  
@show size(X1);  
  
typeof(X1) = IndirectArrays.IndirectArray{ColorTypes.RGB{FixedPointNumbers.N0F8}, 2, UInt8, Matrix{UInt8}, OffsetArrays.OffsetVector{ColorTypes.RGB{FixedPointNumbers.N0F8}}, Vector{ColorTypes.RGB{FixedPointNumbers.N0F8}}}  
size(X1) = (180, 280)
```

Рис. 5: Примеры со считыванием картинки.

Примеры с кластеризацией, метод k средних

Выполнила примеры с кластеризацией, метод k средних:

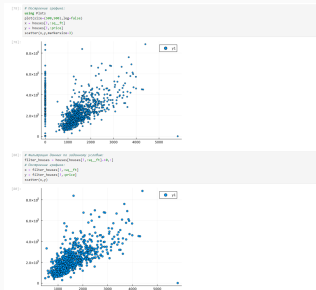


Рис. 6: Примеры с кластеризацией, метод k средних.

Примеры с линейной регрессией

Выполнила примеры с линейной регрессией:

```
[96]: function find_best_fit(xvals,yvals)
      meanx = mean(xvals)
      meany = mean(yvals)
      stdx = std(xvals)
      stdy = std(yvals)
      r = cor(xvals,yvals)
      a = r*stdy/stdx
      b = meany - a*meanx
      return a,b
    end
```

```
[96]: find_best_fit (generic function with 1 method)
```

```
[98]: a,b = find_best_fit(xvals,yvals)
      ymev = a * xvals .+ b
      plot!(xvals,ymev)
```

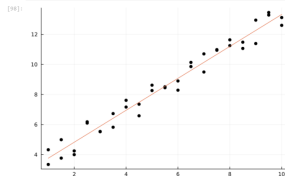


Рис. 7: Примеры с линейной регрессией.

Задания для самостоятельного выполнения

Задание 1

Выполнила первое задание для самостоятельной работы:

```
[118]: using RDatasets
iris = dataset("datasets", "iris")

[118]: 150x5 DataFrame                                     125 rows omitted
      Row  SepalLength  SepalWidth  PetalLength  PetalWidth  Species
      ----  -
      1      5.1         3.5         1.4         0.2  setosa
      2      4.9         3.0         1.4         0.2  setosa
      3      4.7         3.2         1.3         0.2  setosa
      4      4.6         3.1         1.5         0.2  setosa
      5      5.0         3.6         1.4         0.2  setosa
      6      5.4         3.9         1.7         0.4  setosa
      7      4.6         3.4         1.4         0.3  setosa

[120]: iris_features = iris[, 1:4]
iris_matrix = Matrix(iris_features)
println("Размерность матрицы признаков: ", size(iris_matrix))
Размерность матрицы признаков: (4, 150)

[130]: k = 3
result = kmeans(iris_matrix, k)
cluster_labels = assignments(result)
centroids = result.centers

println("Количество кластеров: ", k)
println("Размеры кластеров: ", counts(result))
println("Итераций: ", result.iterations)
println("Общая стоимость (сумма квадратов расстояний): ", round(result.totalcost, digits=2))
Количество кластеров: 3
Размеры кластеров: [50, 61, 39]
Итераций: 2
Общая стоимость (сумма квадратов расстояний): 78.86
```

Рис. 8: Загрузка, кластеризация данных.

Задание 1

Визуализировала:



Рис. 9: Визуализация, добавление центроидов.

Вывела итоговую сводную информацию о кластерах:

```
[144]: # Расчет интервала (суммы взвешенной расстояний до центроидов)
println("Информация о кластерах:")
for i in 1:k
    cluster_points = iris_matrix[:, cluster_labels .+ i]
    cluster_size = size(cluster_points, 2)

    if cluster_size > 0
        # Среднее расстояние до центроидов
        distances = [sqrt(sum((cluster_points[:, j] - centroids[:, i]).^2))
                     for j in 1:cluster_size]
        avg_distance = round(mean(distances), digits=2)
        std_distance = round(std(distances), digits=2)

        println("Кластер $i: $cluster_size образцов, среднее расстояние до центроида: $avg_distance ± $std_distance")
    end
end
```

Информация о кластерах:
Кластер 1: 58 образцов, среднее расстояние до центроида: 0.48 ± 0.27
Кластер 2: 61 образцов, среднее расстояние до центроида: 0.75 ± 0.51
Кластер 3: 39 образцов, среднее расстояние до центроида: 0.73 ± 0.35

Рис. 10: Информация о кластерах.

Выполнила второе задание для самостоятельной работы:

```
[151]: # Генерация данных
N = 1000
X = randn(N, 3)
a0 = rand(1)
y = X * a0 + 0.1 * randn(N)

# МНК-оценка вручную (с добавлением столбца единиц для свободного члена)
X2 = hcat(X, ones(N, 1))
a_mnk = (X2' * X2) \ (X2' * y) # Решение системы (X'X)a = X'y
println("МНК-оценка (ручная): ", a_mnk)

# Сравнение с MultivariateStats.jl (функция 'llsq')
a_llsq = llsq(X, y, bias=true) # bias=true добавляет константу (столбец единиц)
println("МНК-оценка (MultivariateStats.jl, llsq): ", a_llsq)

МНК-оценка (ручная): [0.00144444920711317669, 0.8776963660618514, 0.6533362093380224, 0.5258295482822944]
МНК-оценка (MultivariateStats.jl, llsq): [0.8776963660618513, 0.6533362093380224, 0.5258295482822944, -0.001444449207113177333]
```

Рис. 11: Генерация данных и ручная оценка.

Сопоставила результаты:

```
[163]: # Сравнение с GLM.jl (регулярная линейная регрессия)
# создаем DataFrame для GLM (если не установлен – установите пакет DataFrames)
using DataFrames
df = DataFrame(X1=X[:,1], X2=X[:,2], X3=X[:,3], y=y)
model_glm = lm(@formula(y ~ X1 + X2 + X3), df)
a_glm = coef(model_glm)
println("RRE-оценка (GLM.jl): ", a_glm)

# Проверка соответствия результатов
println("\nСравнение результатов")
println("Разница между ручным RRE и llsq: ", round(norm(a_rnk - a_llsq), digits=3))
println("Разница между ручным RRE и GLM: ", round(norm(a_rnk - a_glm), digits=3))

# Теоретическое объяснение
println("""
Матрица X2 = [1 X] позволяет оценить модель  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$ .
Теоретически, RRE-оценка находится как  $a = (X2' X2)^{-1} X2' y$ .
Это стандартный метод решения overdetermined системы уравнений.
""")

RRE-оценка (GLM.jl): [-0.003444492471317813, 0.8770063668918512, 0.8533362085388224, 0.5258295482822944]

Сравнение результатов:
Разница между ручным RRE и llsq: 1.057
Разница между ручным RRE и GLM: 0.0
Матрица X2 = [1 X] позволяет оценить модель  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$ .
Теоретически, RRE-оценка находится как  $a = (X2' X2)^{-1} X2' y$ .
Это стандартный метод решения overdetermined системы уравнений.
```

Рис. 12: Сопоставление результатов.

Задание 2

Построила линейную регрессию и её график:

```
[105]: # Генерация данных
X = rand(100)
y = 2 .* X + 0.1 * randn(100)

# Регрессия (одно переменная, с константой)
X2 = hcat(ones(100), X)
a_mnk_id = (X2' * X2) \ (X2' * y)
println("Оценки коэффициентов (одномерная регрессия): ", a_mnk_id)

# Построение графика
p = scatter(X, y, label="Данные", title="График регрессии", xlabel="x", ylabel="y")

# Расчеты для линии регрессии
x_line = range(minimum(X), maximum(X), length=100)
y_line = a_mnk_id[1] + a_mnk_id[2] .* x_line

# Добавление линии регрессии
plot!(p, x_line, y_line, label="Линия регрессии", color=:red, linewidth=2)

# Отображение графика
display(p)

Оценки коэффициентов (одномерная регрессия): [-0.0047199675584046474, 2.0250609316052252]
```

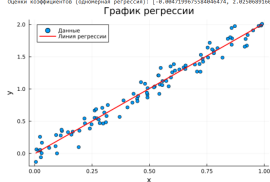


Рис. 13: Линейная регрессия и график.

Построила модель ценообразования биномиальных опционов:

```
[108]: function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Int64)
        h = T / n
        u = exp(r * h + sigma * sqrt(h))
        d = exp(r * h - sigma * sqrt(h))
        p_star = (exp(r * h) - d) / (u - d)

        # Инициализация массива
        path = Vector{Float64}(undef, n+1)
        path[1] = S

        for i in 2:(n+1)
            if rand() < p_star
                path[i] = path[i-1] * u
            else
                path[i] = path[i-1] * d
            end
        end

        return path
    end

# Параметры
S = 100.0
T = 1.0
n = 10000
sigma = 0.3
r = 0.08;
```

Рис. 14: Модель ценообразования биномиальных опционов.

Задание 3

Построила график траектории цены акции:

```
[178]: # Генерируем одну траекторию
path_single = crataPath(S, r, sigma, T, n)

# Строим график
xvals = range(0, T, length=n+1)
plot(xvals, path_single,
      label="Одна траектория",
      title="Траектория цены акции (Биномиальная модель)",
      xlabel="Время (годы)",
      ylabel="Цена акции",
      linewidth=1.5,
      color="blue")

display(plot())
```

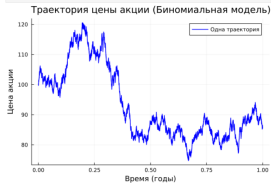


Рис. 15: График траекторий цены акции.

Построила график для 10 траекторий с помощью createPath:

```
[171]: # Генерируем 10 траекторий
num_paths = 10
paths = [createPath(S, r, sigma, T, n) for _ in 1:num_paths]

# Создаем для траекторий на одном графике
p = plot(title="10 траекторий цены акции", xlabel="Время (годы)", ylabel="Цена акции", legend=false)

for path in paths
    plot!(p, x_vals, path, linewidth=0.8, alpha=0.7)
end

display(p)
```

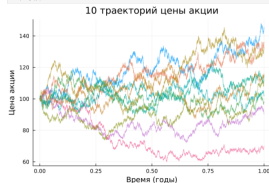


Рис. 16: График для различных траекторий цены.

Задание 3

Построила график для 10 траекторий с помощью createPath и распараллелила с помощью threads:

```
[174]: paths_parallel = Vector(Vector(Float64))(undef, num_paths)

Threads.@threads for i in 1:num_paths
    paths_parallel[i] = createPath(S, r, sigma, T, n)
end

# Строим график
p_parallel = plot(title="10 траекторий (параллельно)", xlabel="Время (годы)", ylabel="Цена акции", legend=false)

for path in paths_parallel
    plot!(p_parallel, x_vals, path, linewidth=0.5, alpha=0.7)
end

display(p_parallel)
```

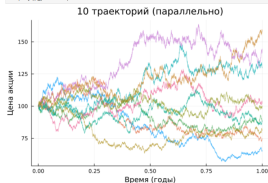


Рис. 17: Параллельное вычисление траекторий.

Заключение

Освоила специализированные пакеты Julia для обработки данных.